LPS · lps_synthesis

Underwater Acoustic Simulator

A practical guide to the library

From the sound speed profile to the `.wav` file:
environments, ships, sonars, dynamics and complete scenarios,
with runnable examples.

This documentation was generated by artificial intelligence.
Text, examples and figures were produced by Claude (Anthropic) from the library's source code. The examples were executed and the outputs shown are real, but the content should be reviewed before being used as a reference.

Repository `musas` · library `lps_libraries/acoustic_synthesis`
October 2026

Contents

0 Introduction **1**

0.1 The architecture on one page . . . 1
0.2 Runtime environment . . . 2
0.3 Conventions . . . 2

1 Sound speed profile (SSP) **4**

1.1 Defining the SSP point by point . . . 4
1.2 Available seabed types . . . 5
1.3 Layers with arbitrary properties . . . 5
1.4 Saving, loading and generating profiles . . . 6
1.5 Plotting . . . 7

2 A complete acoustic environment **8**

2.1 Background noise: `Environment` . . . 8
2.1.1 Querying the PSD and generating noise . . . 9
2.2 Propagation channel . . . 10
2.2.1 `QueryConfig` parameters . . . 11
2.2.2 Using the channel . . . 11

3 Ships and noise sources **13**

3.1 Broadband noise: ship type and propulsion . . . 13
3.2 Creating the ship . . . 15
3.3 Narrowband noise . . . 15
3.3.1 Ready-made and custom modulations . . . 16
3.4 Generating the source signals . . . 17

4 Sonars and sensors **19**

4.1 Hydrophone . . . 19
4.2 Arrays: line and cylindrical . . . 19
4.3 Custom sonar . . . 20
4.4 Gain chain and parameter choice . . . 21

5 Simulation dynamics **23**

5.1 Kinematic state . . . 23
5.2 Moving elements . . . 23
5.3 Controlling the dynamics over time . . . 24
5.4 Ready-made scenario plots . . . 26
5.5 Standardized dynamics . . . 26

6 Complete scenario and `.wav` export **28**

6.1 Setup . . . 28

7 Catalogs and datasets **33**
7.1 Ships from the catalog 33
7.2 Real locations and dataset generation 33

A Quick reference **35**

B Known pitfalls **36**

CHAPTER 0

Introduction

This manual shows, through examples, how to use the `lps_synthesis` library (in `lps_libraries/acoustic_synthe` to build passive underwater acoustic simulations and generate the signals a sonar would receive. The mathematical formulation of the models is documented in a separate paper. The focus here is *how to write the code*: which classes to instantiate, which parameters matter, and how to combine the building blocks.

Every code snippet in this document was extracted automatically from scripts in `docs/manual/examples_en/`, which were run end to end while generating the PDF. The figures and the OUTPUT boxes are real results of those runs.

0.1 The architecture on one page

A simulation is made of five independent blocks that come together in a `Scenario` object (Figure 1):

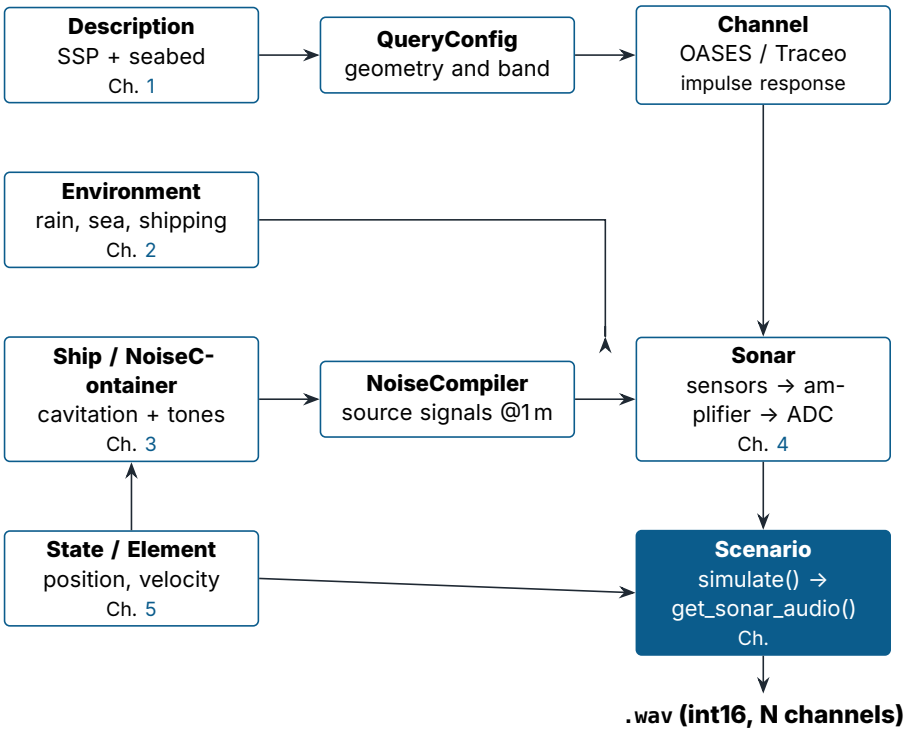


Figure 1: The library's building blocks and the path to the `.wav` file. Arrows point from producer to consumer.

For each (source, sensor) pair, the signal generated at the source (referred to 1 m) goes through source Doppler, the channel impulse response at the instantaneous range, sensor Doppler, and the sensor's sensitivity and directivity. The environment's background noise, signal conditioning and the A/D converter are then applied to the sum.

Module	Contents
`lps_utils.quantities`	Quantities with units: `Distance`, `Speed`, `Frequency`, `Time`, `Sensitivity`, `Bearing`…
`lps_synthesis.propagation`	`Description` (SSP), layers, `QueryConfig`, `Channel`, `Oases`/`Traceo` models
`lps_synthesis.environment`	`Environment` (background noise), oceanographic data prospectors
`lps_synthesis.scenario`	`Ship`, noise sources, `Sonar`, dynamics, `Scenario`
`lps_synthesis.database`	Catalogs for generating datasets (ships, dynamics, locations)

0.2 Runtime environment

The library depends on the repository's other libraries (`lps_utils`, `lps_sp`) and, to compute channels, on OASES (`oasp`) or TRACEO. The simplest way to get everything is the `labsonar/sonar` container, where OASES lives in `/opt/oases/bin`.

terminal

```
# install the libraries in development mode (repository root)
./manager install

# or, without installing, point PYTHONPATH at the sources:
export PYTHONPATH=lps_libraries/acoustic_synthesis/src:lps_libraries/utils/src:\
lps_libraries/signal_processing/src
export PATH=/opt/oases/bin:$PATH          # required by lps_model.Oases()

# run one example from this manual inside the container
DOC_LANG=en docs/manual/run_example.sh ex01_ssp.py

# rebuild the whole manual (runs the examples and compiles the PDF)
docs/manual/build.sh --lang en
```

Channel cache Computing a channel with OASES can take minutes. Each `Channel` therefore saves its impulse response to disk, in a file named after the configuration's *hash*. The default directory is `/data/channel` (`lps_synthesis.propagation.channel.DEFAULT_DIR`), and the examples in this manual use `docs/manual/build_en/channel`. From the second run on, the channel is just loaded.

Always give each channel a unique `hash_id` In this version, the *hash* of `QueryConfig` is the same for every configuration (see Appendix B). Two different channels created in the same `channel_dir` without a `hash_id` get the same file name, and the second one *silently* loads the response of the first. Pass `hash_id="unique_name"` when creating each `Channel`, and change the `hash_id` (or delete the old file) whenever you change the SSP or the geometry.

0.3 Conventions

- **Explicit units.** Every physical parameter is an object from `lps_utils.quantities`: write `lps_qty.Distance.m(30)`, never `30`. Arithmetic respects units, and conversion is done with `get_m()`, `get_kt()`, `get_hz()` and so on.
- **Coordinate system.** Horizontal plane with x pointing east and y pointing north. `Bearing`

angles can be built with a mathematical reference (`eccw`: from east, counter-clockwise) or a nautical one (`ncw`: from north, clockwise).

- **Ship frame.** Relative positions (`rel_position`) of sources and sensors are given in the element's frame: x towards the bow and y towards port. They rotate with the heading.
- **Levels.** Sources in dB re 1 µPa @1 m; PSDs in dB re 1 μPa^2/Hz; sensitivities in dB re 1 V/µPa.
- **Import aliases.** The examples always use the same aliases:

examples_en/ex01_ssp.py

```
import lps_utils.quantities as lps_qty
import lps_synthesis.propagation.layers as lps_layer
import lps_synthesis.propagation.channel_description as lps_desc
```

examples_en/ex01_ssp.py

```
    depth = lps_qty.Distance.m(30)            # also: .km(), .ft(), .nm(), .yd()
    speed = lps_qty.Speed.m_s(1505)           # also: .kt(), .km_h()
    freq = lps_qty.Frequency.khz(2)           # also: .hz(), .rpm()

    print(depth.get_ft(), speed.get_kt(), freq.get_hz())
    print(depth + lps_qty.Distance.km(0.1))   # arithmetic respects units
```

OUTPUT

```
98.43 ft | 2925.49 kt | 2000 Hz
130.0 m
```

CHAPTER 1

Sound speed profile (SSP)

The propagation medium is described by a `Description` object (module `channel_description` of `lps_synthesis.propagation`). It is an ordered list of layers indexed by the depth of their top: air at the surface (by default), water layers with their sound speed and, finally, the seabed.

1.1 Defining the SSP point by point

The `add(depth, layer)` method accepts three kinds of layer:

- a `Speed`, which becomes a `Water` layer with that sound speed;
- a `SeabedType` (or its alias `BottomType`), which becomes a seabed with tabulated properties;
- any `AcousticalLayer`, for full control over the properties.

examples_en/ex01_ssp.py

```
desc = lps_desc.Description()

# Water layers: depth -> sound speed
desc.add(lps_qty.Distance.m(0),  lps_qty.Speed.m_s(1512))
desc.add(lps_qty.Distance.m(20), lps_qty.Speed.m_s(1513))   # mixed layer
desc.add(lps_qty.Distance.m(40), lps_qty.Speed.m_s(1508))
desc.add(lps_qty.Distance.m(70), lps_qty.Speed.m_s(1497))   # thermocline
desc.add(lps_qty.Distance.m(100), lps_qty.Speed.m_s(1492))

# Seabed: a predefined type (properties from Table 1.3 of Jensen et al.)
desc.add(lps_qty.Distance.m(120), lps_layer.BottomType.SAND)
```

The description can be printed, iterated over and queried at any depth. `get_speed_at` interpolates linearly between water layers:

examples_en/ex01_ssp.py

```
print(desc)
print("c(10 m) =", desc.get_speed_at(lps_qty.Distance.m(10)))
print("c(55 m) =", desc.get_speed_at(lps_qty.Distance.m(55)))
```

OUTPUT

```
At 0.0: Air layer — pc=340 m/s / ρ=1.225 kg/m^3
At 0.0: Water layer — pc=1512 m/s / ρ=1 g/cm^3
At 20.0: Water layer — pc=1513 m/s / ρ=1 g/cm^3
At 40.0: Water layer — pc=1508 m/s / ρ=1 g/cm^3
At 70.0: Water layer — pc=1497 m/s / ρ=1 g/cm^3
At 100.0: Water layer — pc=1492 m/s / ρ=1 g/cm^3
At 120.0: Sand
c(10 m) = 1512.5 m/s
c(55 m) = 1502.5 m/s
```

> **Note** Adding two layers at the same depth replaces the first one, because layers are indexed by depth. To remove a layer, use `desc.remove(depth)`. The air–sea interface at 0 m is created automatically and can be removed with `remove_air_sea_interface()`.

1.2 Available seabed types

The predefined types follow Table 1.3 of *Computational Ocean Acoustics* (Jensen et al.). The table below was generated by the example from the library itself:

`SeabedType`	c_p (m/s)	c_s (m/s)	α_p (dB/λ)	α_s (dB/λ)	ρ (g/cm^3)
Clay	1500	50	0.2	1.0	1.5
Silt	1575	80	1.0	1.5	1.7
Sand	1650	110	0.8	2.5	1.9
Gravel	1800	180	0.6	1.5	2.0
Moraine	1950	600	0.4	1.0	2.1
Chalk	2400	1000	0.2	0.5	2.2
Limestone	3000	1500	0.1	0.2	2.4
Basalt	5250	2500	0.1	0.2	2.7

1.3 Layers with arbitrary properties

When you have in-situ measurements (water density, a specific sediment), build the layers explicitly:

examples_en/ex01_ssp.py

```
custom = lps_desc.Description()
custom.add(lps_qty.Distance.m(0), lps_layer.Water(
    sound_speed=lps_qty.Speed.m_s(1500),
    density=lps_qty.Density.kg_m3(1025)))
custom.add(lps_qty.Distance.m(50), lps_qty.Speed.m_s(1495))

# Seabed with arbitrary properties (e.g. mud measured in situ)
custom.add(lps_qty.Distance.m(80), lps_layer.AcousticalLayer(
    compressional_speed=lps_qty.Speed.m_s(1550),
    shear_speed=lps_qty.Speed.m_s(100),
    compressional_attenuation=0.5,          # dB/lambda
    shear_attenuation=1.0,                  # dB/lambda
    density=lps_qty.Density.g_cm3(1.6)))

# By default there is an air-sea interface at 0 m; it can be removed:
# custom.remove_air_sea_interface()
```

1.4 Saving, loading and generating profiles

examples_en/ex01_ssp.py

```
desc.save(f"{OUT_DIR}/my_ssp.json")         # readable .json or .pkl (pickle)
same_desc = lps_desc.Description.load(f"{OUT_DIR}/my_ssp.json")

rand_desc = lps_desc.Description.get_random(seed=7)   # random synthetic profile
default_desc = lps_desc.Description.get_default()     # 1500 m/s, 50 m, chalk
```

The JSON format is readable and easy to edit by hand:

OUTPUT


```
{
    "air_sea": true,
    "layers": {
        "0.0": {
            "type": "Water",
            "speed": 1512.0,
            "value": "None"
        },
        "20.0": {
            "type": "Water",
            "speed": 1513.0,
            "value": "None"
        },
        "40.0": {
            "type": "Water",
            "speed": 1508.0,
            "value": "None"
        },
        "70.0": {
            "type": "Water",
            "speed": 1497.0,
            "value": "None"
        },
        "100.0": {
            "type": "Water",
            "speed": 1492.0,
            "value": "None"
        },
        "120.0": {
            "type": "Seabed",
            "speed": 1650.0000000000002,
            "value": "SAND"
        }
    }
}
```


JSON vs. pickle JSON only stores the sound speed of each `Water` layer and the *name* of the seabed type. Custom densities and generic layers (`AcousticalLayer`) are not restored: `load` raises `NotImplementedError` for them. To save a complete description, use the `.pkl` extension.

1.5 Plotting

`desc.export_ssp("ssp.png")` produces a simple plot. For custom figures, just iterate over the description:

examples_en/ex01_ssp.py

```
def plot_ssp(description, label):
    depths, speeds = [], []
    for d, layer in description:                    # iterates (depth, layer)
        if isinstance(layer, lps_layer.Water):
            depths.append(d.get_m())
            speeds.append(layer.get_compressional_speed().get_m_s())
    plt.plot(speeds, depths, marker="o", label=label)

plt.figure(figsize=(5, 4.5))
plot_ssp(desc, "Manual")
plot_ssp(same_desc, "Reloaded (JSON)")
plt.gca().invert_yaxis()
plt.xlabel("Sound speed (m/s)")
plt.ylabel("Depth (m)")
plt.legend()
plt.savefig(fig_path("ex01_ssp.png"))
plt.close()
```

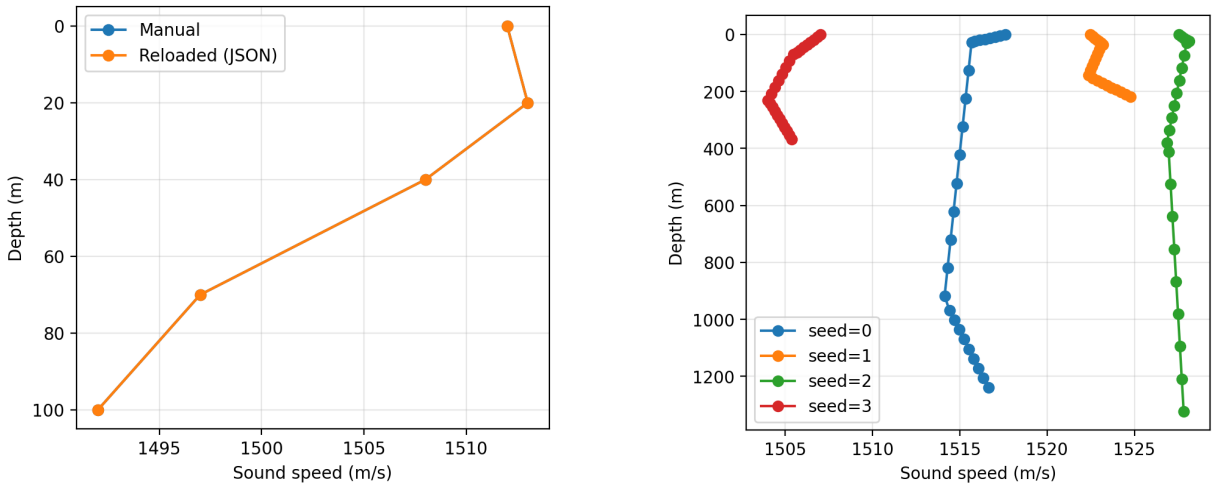


Figure 1.1: Left: manually defined SSP (and the same SSP reloaded from JSON). Right: profiles from `Description.get_random(seed)`, which draws the depth, the profile type (positive, negative or isovelocity), mixed layer, thermocline and seabed type.

CHAPTER 2

A complete acoustic environment

The "environment" of a simulation has two independent parts:

1. the **background noise** (`Environment`), added directly at each sensor;
2. the **propagation channel** (`Channel`), which transforms the signal of each source according to the SSP, the seabed and the source–sensor range.

2.1 Background noise: `Environment`

The model combines four components, based on the curves in Ch. 7 of Hodges, *Underwater Acoustics*: turbulence, rain, sea state and distant shipping.

examples_en/ex02_environment.py

```
environment = lps_env.Environment(
    rain_value=lps_env.Rain.LIGHT,          # NONE, LIGHT, MODERATE, HEAVY, VERY_HEAVY
    sea_value=lps_env.Sea.STATE_3,          # STATE_0 ... STATE_6
    shipping_value=lps_env.Shipping.LEVEL_3,  # NONE, LEVEL_1 ... LEVEL_7
    seed=42,                                # reproducibility of the generated noise
)
```

Each component accepts either an `enum` value or a real number within its range. In the latter case, the PSD is interpolated between the tabulated levels:

Parameter	Enum	Range	Levels
`rain_value`	`Rain`	0–4	NONE, LIGHT (1 mm/h), MODERATE (5), HEAVY (10), VERY_HEAVY (100)
`sea_value`	`Sea`	0–6	STATE_0 ...STATE_6 (Douglas scale)
`shipping_value`	`Shipping`	0–7	NONE, LEVEL_1 ...LEVEL_7

examples_en/ex02_environment.py

```
# Continuous values interpolate between the tabulated levels
env_float = lps_env.Environment(rain_value=0.5, sea_value=2.3, shipping_value=4.7)

# Randomly drawn environment (useful for building datasets)
env_rand = lps_env.Environment.random(seed=3)

# Global attenuation (dB) applied to the whole background noise
env_quiet = lps_env.Environment(lps_env.Rain.NONE, lps_env.Sea.STATE_1,
                                lps_env.Shipping.LEVEL_1, global_attenuation_db=10)
```

OUTPUT

```
Rain[light], Sea[sea state 3], Shipping[shipping noise level 3]
```

```
Rain[0.5], Sea[2.3], Shipping[4.7]
Rain[1.0], Sea[3.3], Shipping[2.6]
```

2.1.1 Querying the PSD and generating noise

examples_en/ex02_environment.py

```
freqs, psd_db = environment.to_psd()                # theoretical PSD (dB re 1 μ
↪ Pa²/Hz)

fs = lps_qty.Frequency.khz(16)
noise = environment.generate_bg_noise(n_samples=int(10 * fs.get_hz()),
                                      fs=fs.get_hz())  # time series in μPa

est_freqs, est_psd = lps_bb.psd(noise, fs=fs.get_hz(), window_size=4096)

environment.save_plot(fig_path("ex02_env_save_plot.png"))   # ready-made plot
```

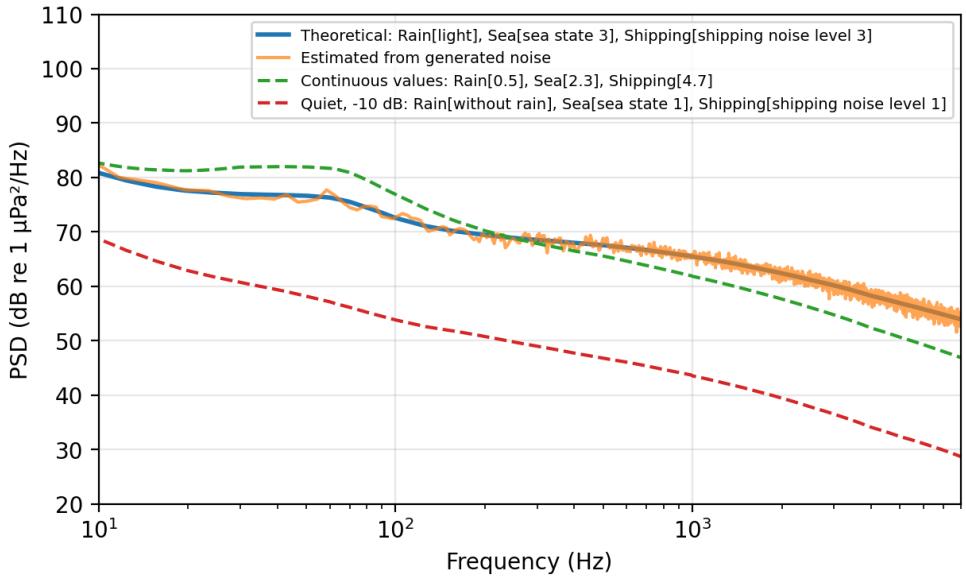


Figure 2.1: Theoretical PSD (`to_psd`) and the PSD estimated from 10 s of `generate_bg_noise`, plus two other configurations.

Warning `generate_bg_noise` takes `fs` as a `float` in Hz (`fs.get_hz()`), not as a `Frequency`. Noise is drawn from the object's internal generator (`seed`). With the same `seed`, the same sequence of calls always produces the same noise.

Each component can also be inspected on its own:

examples_en/ex02_environment.py

```
for component in [lps_env.Rain.HEAVY, lps_env.Sea.STATE_4, lps_env.Shipping.LEVEL_5]:
    f, p = component.get_psd()       # each component can be queried on its own
    plt.semilogx(f, p, label=str(component))
```

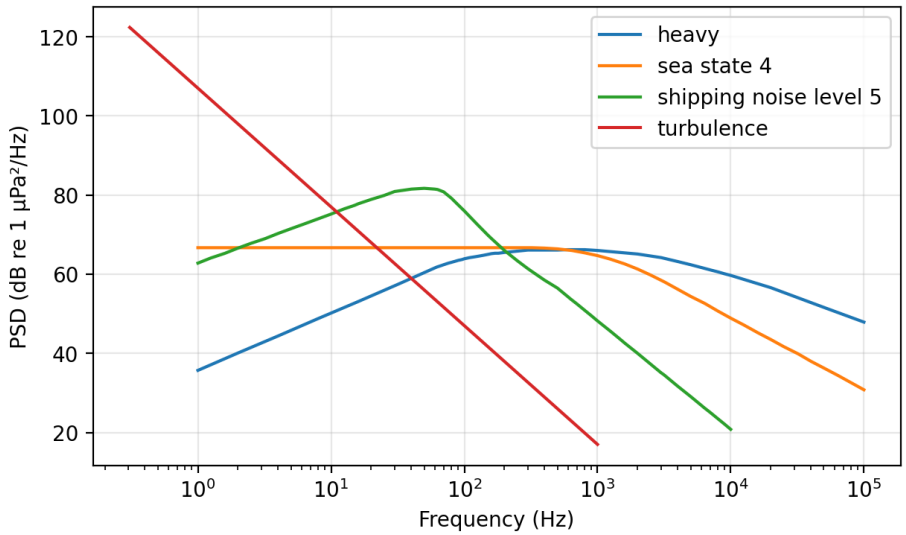


Figure 2.2: Individual background noise components.

2.2 Propagation channel

A channel is defined by three objects:

1. a `Description` (Chapter 1);
2. a `QueryConfig`, which says *where* and *in which band* to compute the response;
3. a propagation model: `lps_model.Oases()` (default) or `lps_model.Traceo()`.

examples_en/ex02_environment.py

```
def build_channel() -> lps_channel.Channel:
    """ Shallow-water channel (120 m, sandy bottom) used in the manual's examples. """

    desc = lps_desc.Description()
    desc.add(lps_qty.Distance.m(0),   lps_qty.Speed.m_s(1512))
    desc.add(lps_qty.Distance.m(20),  lps_qty.Speed.m_s(1513))
    desc.add(lps_qty.Distance.m(40),  lps_qty.Speed.m_s(1508))
    desc.add(lps_qty.Distance.m(70),  lps_qty.Speed.m_s(1497))
    desc.add(lps_qty.Distance.m(100), lps_qty.Speed.m_s(1492))
    desc.add(lps_qty.Distance.m(120), lps_layer.BottomType.SAND)

    query = lps_model.QueryConfig(
        description=desc,
        sensor_depth=lps_qty.Distance.m(40),                # sonar depth
        source_depths=[lps_qty.Distance.m(5),               # source depths
                       lps_qty.Distance.m(10)],             #  (ship drafts)
        max_distance=lps_qty.Distance.m(300),               # maximum computed range
        max_distance_points=150,                            # range resolution
        frequency_range=(lps_qty.Frequency.hz(10),          # computed band
                         lps_qty.Frequency.khz(4)),
        sample_frequency=lps_qty.Frequency.khz(16),
    )

    return lps_channel.Channel(
        query=query,
```

```
        model=lps_model.Oases(),          # or lps_model.Traceo()
        channel_dir=CHANNEL_DIR,          # cache: default is /data/channel
        hash_id="manual_shallow_water",   # unique name in the cache (see Appendix B)
    )
```

2.2.1 `QueryConfig` parameters

Parameter	Default	Meaning
`description`	—	Propagation medium (SSP + seabed).
`sensor_depth`	—	Receiver depth. **This is what sets the sonar depth in the simulation.**
`source_depths`	2, 4, ..., 20 m	Source depths to compute. The list becomes a regular grid whose step is the GCD of the differences ([4, 10] becomes 4, 6, 8, 10). During the simulation, each source uses the computed depth closest to its draft.
`max_distance`	1 km	Maximum range. Longer ranges reuse the response of the last point.
`max_distance_points`	range/5 m	Range resolution. The step is rounded to 1, 2 or 5×10^n m.
`frequency_range`	full band	Restricts the computed frequencies (e.g. 10 Hz–4 kHz). The response is zero outside it.
`sample_frequency`	16 kHz	Impulse response sample rate. If the simulation uses a different rate, the response is resampled.

Computational cost The number of computed frequencies is $N_{\mathrm{FFT}}/2$, with $N_{\mathrm{FFT}} = 2^{\lceil \log_2(2\, r_{\max} f_s/1500)\rceil}$. Doubling the range or the sample rate doubles the OASES workload, which also scales with the number of depths and ranges. The channel in this chapter (300 m, 10 Hz–4 kHz, 2 depths, 151 ranges) took about 7 minutes (404 s) on the first run, and later runs just read the cache in under a second. Restricting `frequency_range` to the band of interest is the most effective way to cut the time.

2.2.2 Using the channel

examples_en/ex02_environment.py

```
t0 = time.time()
channel = build_channel()          # 1st run: runs OASES; afterwards: reads the cache
print(f"Channel ready in {time.time() - t0:.1f} s")

ir = channel.get_ir()              # lps_synthesis.propagation.TemporalResponse
print("h_t_tau:", ir.h_t_tau.shape, "(depths, ranges, samples)")
ir.print_as_image(fig_path("ex02_channel_ir.png"), depth_index=0)

# Propagating a signal: source at 5 m depth, moving away from 50 m to 250 m
pulse = np.zeros(int(2 * fs.get_hz()))
pulse[::int(0.25 * fs.get_hz())] = 1e6          # impulse train (µPa @1 m)
received = channel.propagate(input_data=pulse,
                             source_depth=lps_qty.Distance.m(5),
                             distance=[lps_qty.Distance.m(50),
↪ lps_qty.Distance.m(250)],
                             sample_frequency=fs)
```

OUTPUT

```
Depths: ['5.0 m', '10.0 m']
Ranges: 151 points, from 0.0 km to 0.3000000142492354 km
h_t_tau: (2, 151, 8192) (depths, ranges, samples)
```

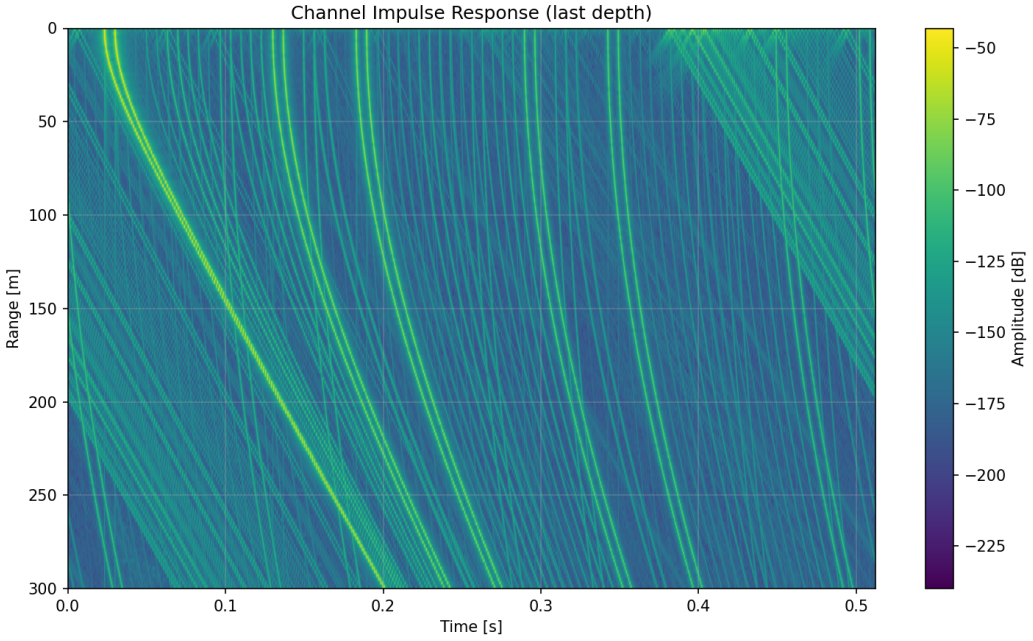


Figure 2.3: `ir.print_as_image()`: impulse response (in dB) for the first source depth, as a function of range and delay.

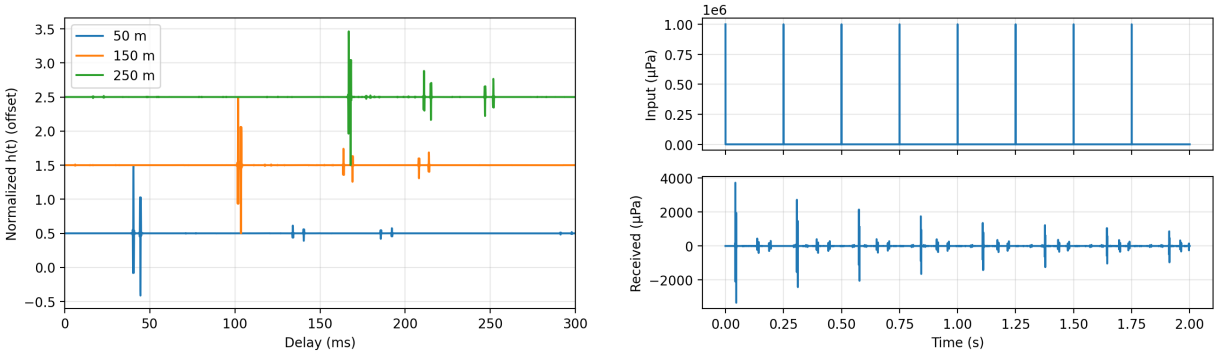


Figure 2.4: Left: impulse response cuts at three ranges (direct, surface- and bottom-reflected arrivals). Right: `channel.propagate` applied to an impulse train whose source moves away from 50 to 250 m.

Channels for real locations To use real oceanographic data (ETOPO bathymetry, WOA18 SSPs, SHOM sediments), the `AcousticSiteProspector` class (`lps_synthesis.environment.acoustic_site`) builds the `Description` and the `Environment` from latitude, longitude and season. It requires the datasets in `/data/ambiental`. Chapter 7 shows the shortcut through `AcousticScenario`.

CHAPTER 3

Ships and noise sources

In the library, a ship is a *container* of point sources (`NoiseContainer`) that moves as a single element. All the sources of a ship share the hull's dynamics and depth (the draft), but each one can sit at a different position on the hull.

Class	Role
`ShipType`	Ship type: spectral model (JOMOPANS-ECHO) and typical parameter ranges
`CavitationNoise`	Propeller broadband noise, modulated by the shaft rotation
`NarrowBandNoise`	Tonal components (machinery, generators, active sonar…)
`Ship`	`NoiseContainer` with propulsion and draft
`NoiseCompiler`	Generates the signals of all sources, grouped by position

3.1 Broadband noise: ship type and propulsion

The broadband spectrum of each type follows the JOMOPANS-ECHO model and depends on length and speed:

examples_en/ex03_ship.py

```
freqs, psd = lps_noise.ShipType.BULKER.to_psd(
    fs=fs,
    lenght=lps_qty.Distance.m(200),     # ship length
    speed=lps_qty.Speed.kt(12))         # speed (default: reference speed)
```

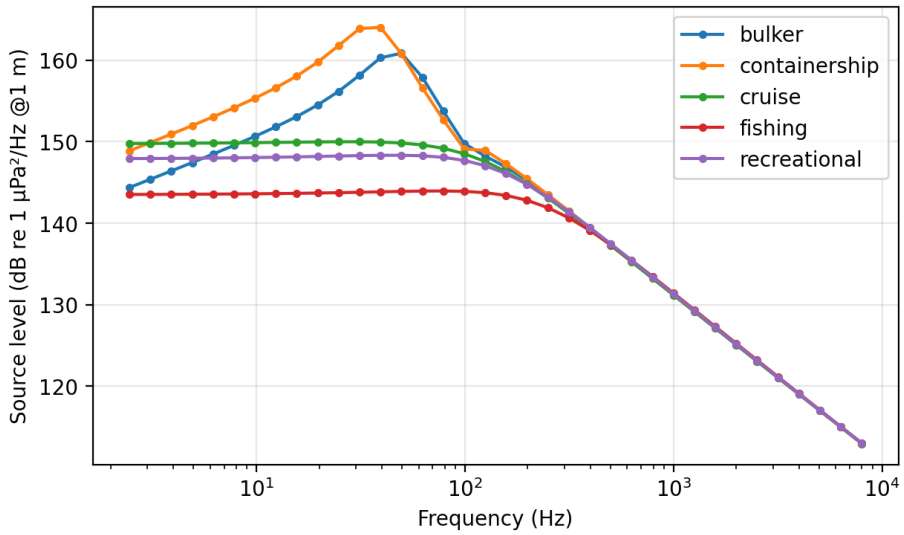


Figure 3.1: `ShipType.to_psd(fs)` at each type's reference speed.

The propulsion (`CavitationNoise`) generates this broadband noise at every simulation step, using the instantaneous speed, and modulates it with the harmonics of the shaft rotation and of

the blade-passing frequency. All parameters are optional. Omitted ones are drawn (with `seed`) within the typical ranges for the ship type.

examples_en/ex03_ship.py

```
propulsion = lps_noise.CavitationNoise(
    ship_type=lps_noise.ShipType.BULKER,
    n_blades=4,                                  # number of propeller blades
    n_shafts=1,                                  # number of shafts
    length=lps_qty.Distance.m(180),
    cruise_speed=lps_qty.Speed.kt(12),           # cruising speed
    cruise_rotacional_frequency=lps_qty.Frequency.rpm(90),   # RPM at cruise
    max_speed=lps_qty.Speed.kt(16),
    seed=1,
)
# Omitted parameters are drawn consistently with the ship type:
random_propulsion = lps_noise.CavitationNoise(ship_type=lps_noise.ShipType.TANKER,
↪ seed=7)
```

examples_en/ex03_ship.py

```
    for p in [propulsion, random_propulsion]:
        print(f"{p.ship_type}: {p.n_blades} blades, {p.n_shafts} shaft(s),
↪ L={p.length}, "
              f"v_cruise={p.cruise_speed}, rot={p.cruise_rotacional_frequency}, "
              f"cavitation threshold={p.cavitation_threshold}")
```

OUTPUT

```
bulker: 4 blades, 1 shaft(s), L=180 m, v_cruise=12 kt, rot=90 RPM, cavitation threshold
    ↪ =22.837112788046444 RPM
tanker: 5 blades, 2 shaft(s), L=248.5 m, v_cruise=11.6 kt, rot=72.9 RPM, cavitation threshold
    ↪ =24.022963422535017 RPM
```

The rotation rate is estimated from the speed, and the modulation index grows above the cavitation threshold. The steps can be run separately:

examples_en/ex03_ship.py

```
broadband, speeds = propulsion.generate_broadband_noise(fs)
modulated, modulating = propulsion.modulate_noise(broadband=broadband,
                                                  speeds=speeds, fs=fs)
rpm = propulsion.estimate_rpm(speeds)                 # RPM estimated at each step
```

OUTPUT

```
speed=10.0 kt -> rotation=76.5 rpm (BPF = 5.10 Hz)
```

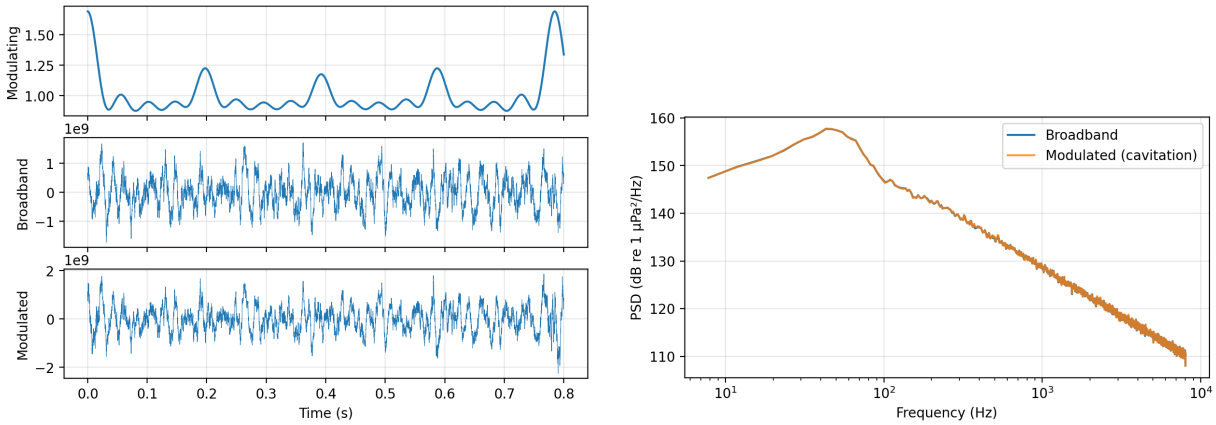


Figure 3.2: Cavitation: modulating signal (rotation harmonics), broadband and modulated signal (left), and the corresponding PSDs (right).

3.2 Creating the ship

examples_en/ex03_ship.py

```
ship = lps_noise.Ship(
    ship_id="Bulker",
    propulsion=propulsion,                  # mandatory in this version (see note)
    draft=lps_qty.Distance.m(10),           # draft = depth of the sources
    initial_state=lps_dynamic.State(
        position=lps_dynamic.Displacement(lps_qty.Distance.m(-150),
↪ lps_qty.Distance.m(50)),
        velocity=lps_dynamic.Velocity(lps_qty.Speed.kt(10), lps_qty.Speed.kt(0))),
    seed=1,
)
# Shortcut: ship with propulsion and draft drawn for a given type
quick_ship = lps_noise.Ship.by_type(lps_noise.ShipType.FISHING, seed=3)
```

Always pass `propulsion` In this version, `Ship(..., propulsion=None)` fails with an `AttributeError`: the constructor draws a propulsion but then reads `propulsion.ship_type` from the argument. Use `Ship.by_type(...)` or pass a `CavitationNoise`. Also, `Ship` overwrites `initial_state.max_speed` with `propulsion.max_speed`. If `initial_state` is omitted, the ship starts at $x = -5\,\mathrm{s} \times v_{\mathrm{cruise}}$, heading east at cruising speed.

3.3 Narrowband noise

Each tonal component is a `NarrowBandNoise` added with `add_source`. The general model is

$$V(t) = A\,\big(1 + \varepsilon(t)\big)\cos\!\big(2\pi f_0 t + \varphi(t)\big), \qquad A = \sqrt{2}\cdot 10^{L/20},$$

where L is `amp_db_p_upa`, the RMS level in dB re 1 µPa @1 m.

examples_en/ex03_ship.py

```
# Pure tone: V(t) = A cos(2 pi f0 t)
ship.add_source(lps_noise.NarrowBandNoise(
```

```
        frequency=lps_qty.Frequency.hz(60),          # e.g. power grid / generator
        amp_db_p_upa=178))                           # level in dB re 1 µPa @1 m

    # Tone at another position on the hull (coordinates in the ship's frame)
    ship.add_source(lps_noise.NarrowBandNoise(
        frequency=lps_qty.Frequency.hz(420),
        amp_db_p_upa=168,
        rel_position=lps_dynamic.Displacement(lps_qty.Distance.m(40),     # 40 m forward
                                              lps_qty.Distance.m(0))))

    # Tone with slow random frequency drift (real machinery)
    ship.add_source(lps_noise.NarrowBandNoise.with_brownian_fm_modulation(
        frequency=lps_qty.Frequency.hz(1200),
        amp_db_p_upa=162,
        freq_std=lps_qty.Frequency.hz(0.02),         # random-walk step (per sample)
        amp_std=1e-4,
        seed=5))
```

`rel_position` places the source at another point on the hull. The effect shows up as small delay and range differences between sources of the same ship and, with a sensor array, in the direction of arrival. The depth is still the ship's draft.

3.3.1 Ready-made and custom modulations

examples_en/ex03_ship.py

```
    nb_am = lps_noise.NarrowBandNoise.with_sine_am_modulation(
        frequency=lps_qty.Frequency.khz(2), amp_db_p_upa=150,
        am_freq=lps_qty.Frequency.hz(0.5), am_depth=1)

    nb_fm = lps_noise.NarrowBandNoise.with_sine_fm_modulation(
        frequency=lps_qty.Frequency.khz(2), amp_db_p_upa=150,
        oscilation_freq=lps_qty.Frequency.hz(0.5),
        deviation_freq=lps_qty.Frequency.hz(500))

    nb_chirp = lps_noise.NarrowBandNoise.with_fm_chirp(     # e.g. active sonar
        amp_db_p_upa=150,
        start_frequency=lps_qty.Frequency.khz(2),
        end_frequency=lps_qty.Frequency.khz(4),
        tx_interval=lps_qty.Time.s(2),                      # repetition period
        tx_duration=lps_qty.Time.s(0.8))                    # pulse length

    nb_brownian = lps_noise.NarrowBandNoise.with_brownian_modulation(
        frequency=lps_qty.Frequency.khz(2), amp_db_p_upa=150,
        amp_std=1e-4, phase_std=0.02, seed=1)

    # Arbitrary modulations: V(t) = A (1 + eps(t)) cos(2 pi f0 t + phi(t))
    nb_custom = lps_noise.NarrowBandNoise(
        frequency=lps_qty.Frequency.khz(2), amp_db_p_upa=150,
        epsilon_fn=lambda t: -0.9 * (np.mod(t, 1.0) > 0.5),       # on/off every 0.5 s
        phi_fn=lambda t: 2 * np.pi * 100 * t**2 / 8)               # slow quadratic chirp
```

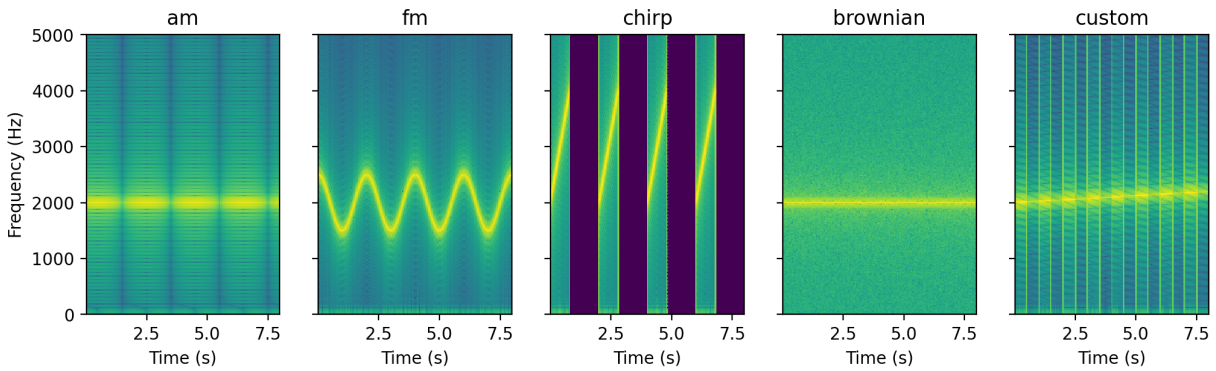


Figure 3.3: Spectrograms of the `NarrowBandNoise` variants (8 s, $f_s = 16$ kHz).

Constructor	Typical use
`NarrowBandNoise(f, L)`	stable tone (power grid, generator, gearbox)
`with_sine_am_modulation`	tone with beating / periodic amplitude modulation
`with_sine_fm_modulation`	tone with periodic frequency variation
`with_fm_chirp`	periodic LFM pulses (active sonar, *pinger*)
`with_brownian_modulation`	random walk in amplitude and phase (broadens the line)
`with_brownian_fm_modulation`	slow random frequency drift
`NarrowBandNoise(f, L, epsilon_fn, phi_fn)`	any modulation: functions of `t` (s) that return arrays

Per-sample random walks In the Brownian variants, `amp_std`, `phase_std` and `freq_std` are *per-sample* deviations, accumulated over the whole signal. After N samples the deviation grows as $\sqrt{N}$: at 16 kHz, 60 s of signal with `amp_std=0.05` makes the amplitude vary by tens of times its nominal value. Use small values (around 10^{-4} for amplitude and 10^{-2} Hz for frequency).

3.4 Generating the source signals

The dynamics must be defined before signals can be generated (Chapter 5). For a standalone ship, just move it. Inside a scenario, `Scenario.simulate` does this for you.

examples_en/ex03_ship.py

```
ship.move(step_interval=lps_qty.Time.s(1), n_steps=30)     # 30 s of trajectory
```

`NoiseCompiler` generates the signal of each source (in µPa, referred to 1 m) and sums the sources at the same point on the hull, since they travel the same path to the sensor:

examples_en/ex03_ship.py

```
compiler = lps_noise.NoiseCompiler([ship], fs=fs)
compiler.show_details()                    # sources grouped by position

for i, (signal, depth, sources) in enumerate(compiler):
    print(i, depth, [s.get_id() for s in sources], signal.shape)

compiler.save_plot(fig_path("ex03_compiler_psd.png"))
```

OUTPUT

```
 - Sources: ['Bulker:cavitation', 'Bulker:NarrowBand [60 Hz]', 'Bulker:NarrowBand [1200 Hz]']
        Depth: 10 m, N samples: 480000
 - Sources: ['Bulker:NarrowBand [420 Hz]']
        Depth: 10 m, N samples: 480000
```

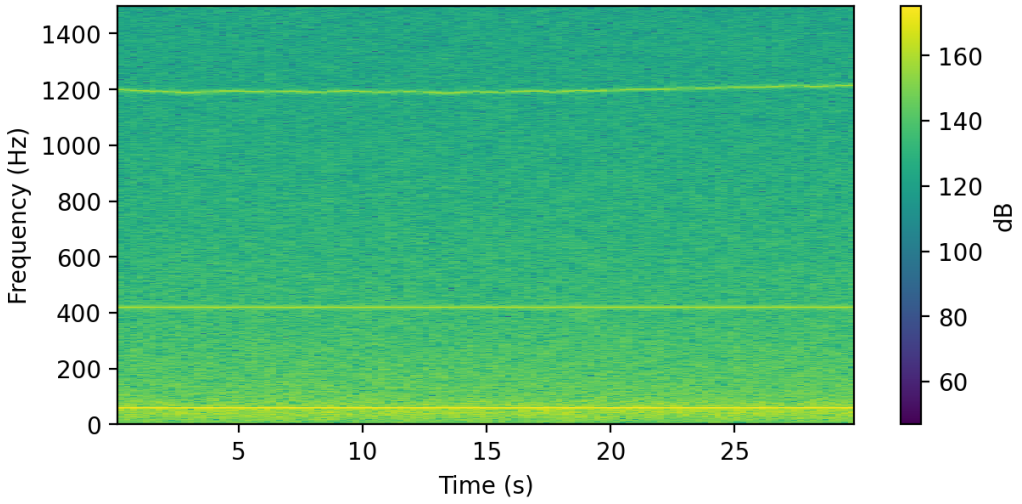


Figure 3.4: Sum of the ship's sources at 1 m: cavitation (broadband, strongest below 200 Hz), tones at 60 and 420 Hz, and the 1200 Hz tone with Brownian drift.

Note `compiler.save_wavs(dir)` saves a normalized `.wav` and a spectrogram for each source group, which helps validate each component before building the scenario.

CHAPTER 4

Sonars and sensors

A `Sonar` is a moving element with a list of sensors (`AcousticSensor`), a shared signal conditioner and an A/D converter. Each sensor converts the received pressure into voltage and is made of:

- a **sensitivity** (`Sensitivity`): `FlatBand` or any subclass (frequency response);
- a **directivity** (`Directivity`): `Omnidirectional`, `Shaded` or a subclass;
- a **position and orientation** relative to the sonar (`rel_position`, `rel_direction`).

4.1 Hydrophone

The simplest case is a single omnidirectional sensor with flat sensitivity:

examples_en/ex04_sonar.py

```
hydrophone = lps_sonar.Sonar.hydrophone(
    sensitivity=lps_qty.Sensitivity.db_v_p_upa(-170),    # dB re 1 V/µPa
    signal_conditioner=lps_sonar.IdealAmplifier(gain_db=40),
    adc=lps_sonar.ADConverter(input_limits=(-5, 5), resolution=16),
    initial_state=lps_dynamic.State(
        position=lps_dynamic.Displacement(lps_qty.Distance.m(0),
↪ lps_qty.Distance.m(0))),
)
```

4.2 Arrays: line and cylindrical

examples_en/ex04_sonar.py

```
# Line array of 8 sensors spaced lambda/2 at 1.5 kHz, along the sonar x axis
line_array = lps_sonar.Sonar.planar(
    n_staves=8,
    spacing=lps_qty.Distance.m(0.5),
    sensitivity=lps_qty.Sensitivity.db_v_p_upa(-170),
)

# Cylindrical array: 16 staves with "Shaded" directivity pointing outwards
cylinder = lps_sonar.Sonar.cylindrical(
    n_staves=16,
    radius=lps_qty.Distance.m(1),
    sensitivity=lps_qty.Sensitivity.db_v_p_upa(-170),
)
```

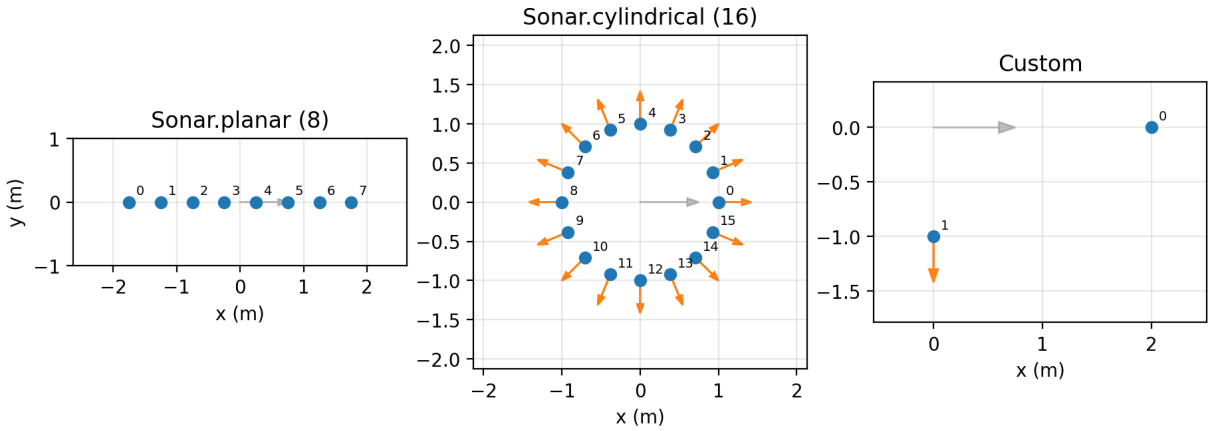


Figure 4.1: Sonar geometries (gray arrow: sonar bow; orange arrows: direction of maximum response of directional sensors). The `planar` array lies along the longitudinal axis, and `cylindrical` uses `Shaded` sensors pointing radially outwards.

4.3 Custom sonar

For any other geometry, build the list of sensors yourself. A frequency response is implemented by subclassing `Sensitivity` and defining `convert(input_data, fs)`, which takes µPa and returns volts:

examples_en/ex04_sonar.py

```
class HighPassSensitivity(lps_sonar.Sensitivity):
    """ Flat sensitivity with a 2nd-order high-pass (e.g. AC coupling). """

    def __init__(self, sensitivity: lps_qty.Sensitivity, cutoff: lps_qty.Frequency):
        super().__init__()
        self.flat = lps_sonar.FlatBand(sensitivity)
        self.cutoff = cutoff

    def convert(self, input_data: np.array, fs) -> np.array:
        fs_hz = fs.get_hz() if isinstance(fs, lps_qty.Frequency) else fs
        sos = sci_sig.butter(2, self.cutoff.get_hz(), btype="highpass", fs=fs_hz,
    ↪ output="sos")
        return sci_sig.sosfilt(sos, self.flat.convert(input_data, fs))
```

examples_en/ex04_sonar.py

```
    sensors = [
        # omnidirectional sensor forward
        lps_sonar.AcousticSensor(
            sensitivity=lps_sonar.FlatBand(lps_qty.Sensitivity.db_v_p_upa(-165)),
            rel_position=lps_dynamic.Displacement(lps_qty.Distance.m(2),
    ↪ lps_qty.Distance.m(0))),
        # directional sensor pointing to starboard (-90°), with a 20 Hz high-pass
        lps_sonar.AcousticSensor(
            sensitivity=HighPassSensitivity(lps_qty.Sensitivity.db_v_p_upa(-165),
                                            lps_qty.Frequency.hz(20)),
            directivity=lps_sonar.Shaded(d=10, m=4),
            rel_direction=lps_qty.RelativeBearing.ccw_deg(-90),
            rel_position=lps_dynamic.Displacement(lps_qty.Distance.m(0),
```

```
                                                lps_qty.Distance.m(-1))),
]

custom_sonar = lps_sonar.Sonar(
    sensors=sensors,
    signal_conditioner=lps_sonar.IdealAmplifier(gain_db=30),
    adc=lps_sonar.ADConverter(input_limits=(-2.5, 2.5), resolution=24),
    initial_state=lps_dynamic.State(
        velocity=lps_dynamic.Velocity(lps_qty.Speed.kt(4), lps_qty.Speed.kt(0))),
)
```

examples_en/ex04_sonar.py

```
fs = lps_qty.Frequency.khz(16)
sensors[1].plot_response(fig_path("ex04_sensor_response.png"), fs=fs)
```

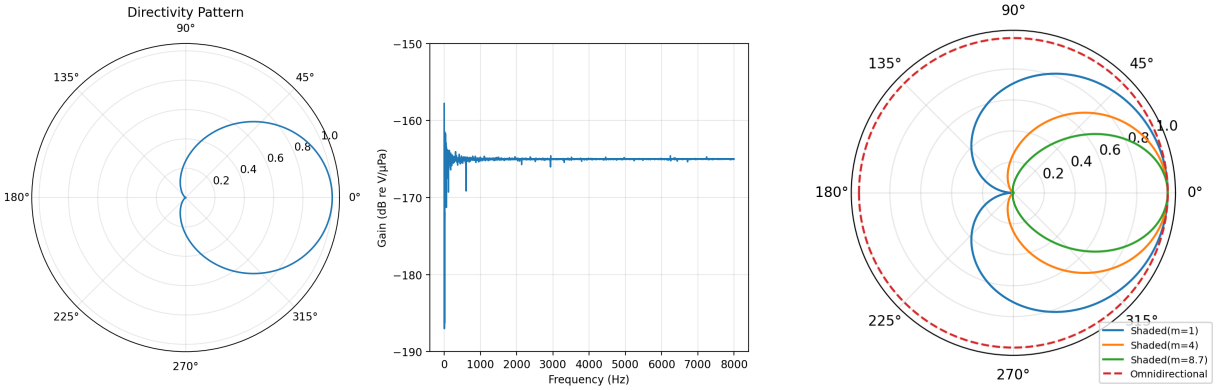


Figure 4.2: Left: `plot_response` of the custom directional sensor (polar pattern and sensitivity with the 20 Hz high-pass). Right: `Shaded` for different orders m.

Warning `Sensitivity.convert` receives `fs` as a `Frequency` during the simulation, but as a `float` in `plot_response`. Custom implementations must accept both, as in the example. Directivity only applies to source signals: the `Environment` background noise is treated as isotropic.

4.4 Gain chain and parameter choice

Each sensor's signal goes through the chain

$$p\ [\mu\text{Pa}] \xrightarrow{\text{Sensitivity}} v\ [\text{V}] \xrightarrow{\text{IdealAmplifier}(G)} v' \xrightarrow{\text{ADConverter}} \text{signed integer}$$

examples_en/ex04_sonar.py

```
p_upa = np.array([1e6])                                  # 120 dB re 1 µPa
v_sensor = lps_sonar.FlatBand(lps_qty.Sensitivity.db_v_p_upa(-170)).convert(p_upa, fs)
v_cond = lps_sonar.IdealAmplifier(gain_db=40).convert(v_sensor, fs)
counts = lps_sonar.ADConverter(input_limits=(-5, 5), resolution=16).apply(v_cond)
```

OUTPUT

```
pressure = 1e+06 µPa -> sensor = 3.162 mV -> amplifier = 316.2 mV -> ADC = 2071 (int16)
```

`ADConverter` clips at `input_limits` (default $\pm 5\,\mathrm{V}$) and outputs `int8`, `int16`, `int32` or `int64`, depending on `resolution`. Choose the sensitivity and gain so that the strongest expected signal (ship noise at the closest point of approach) stays below full scale, and the background noise stays well above 1 LSB. Chapter 6 shows how to measure this.

examples_en/ex04_sonar.py

```
        for name, sonar in [("hydrophone", hydrophone), ("line", line_array),
                            ("cylindrical", cylinder), ("custom", custom_sonar)]:
            sonar.move(lps_qty.Time.s(1), 1)              # places the sensors (see Ch. 5)
            positions = [s[0].position for s in sonar.sensors]
            print(f"{name:<12} {len(sonar.sensors):>2} sensors | "
                  f"x in [{min(p.x.get_m() for p in positions):+.2f}, "
                  f"{max(p.x.get_m() for p in positions):+.2f}] m")
```

OUTPUT

```
hydrophone    1 sensors | x in [+0.00, +0.00] m
line          8 sensors | x in [-1.75, +1.75] m
cylindrical  16 sensors | x in [-1.00, +1.00] m
custom        2 sensors | x in [+0.00, +2.00] m
```

Sonar depth The sensor depth used for propagation does *not* come from the sonar position but from `QueryConfig.sensor_depth`. All sonars in a scenario share the same channel and therefore the same depth. For sonars at different depths, simulate separate scenarios, each with its own channel.

CHAPTER 5

Simulation dynamics

Ships, sonars and standalone sources are `Element`s. Each element keeps its list of states (`State`) over the simulation, one per step of length `step_interval`. Sensors and additional sources are `RelativeElement`s, and their states are derived from the element they belong to.

5.1 Kinematic state

examples_en/ex05_dynamics.py

```
state = lps_dynamic.State(
    position=lps_dynamic.Displacement(lps_qty.Distance.m(-200),     # x: east
                                      lps_qty.Distance.m(80)),      # y: north
    velocity=lps_dynamic.Velocity(lps_qty.Speed.kt(8), lps_qty.Speed.kt(0)),
    acceleration=lps_dynamic.Acceleration(lps_qty.Acceleration.m_s2(0.02),
                                          lps_qty.Acceleration.m_s2(0)),
    max_speed=lps_qty.Speed.kt(12),         # speed is saturated at this value
)

# Vectors can also be created in polar coordinates
v_polar = lps_dynamic.Velocity.polar2d(lps_qty.Speed.kt(10),
                                       lps_qty.Bearing.ncw_deg(45))    # course NE

next_state = state.estimate(lps_qty.Time.s(10))      # kinematics: x + v dt + a dt^2/2
```

OUTPUT

```
t = 0 s : [-200 m, 80 m], [8 kt, 0 kt], [0.02 m/s^2, 0 m/s^2]
t = 10 s: [-157.84444444444443 m, 80.0 m], [8.38876889848812 kt, 0.0 kt], [0.02 m/s^2, 0 m/s^2]
v_polar : 7.0710678118654755 kt, 7.071067811865475 kt | rumo = 45.0° (ncw)
```

`estimate(dt)` integrates constant-acceleration kinematics. If the speed exceeds `max_speed` (50 kt by default), it is saturated and the acceleration is zeroed from then on.

5.2 Moving elements

examples_en/ex05_dynamics.py

```
ship = lps_noise.Ship(
    ship_id="Fishing",
    propulsion=lps_noise.CavitationNoise(lps_noise.ShipType.FISHING, seed=2),
    draft=lps_qty.Distance.m(5),
    initial_state=state)

states = ship.move(step_interval=lps_qty.Time.s(1), n_steps=60)  # list of State
print(len(states), ship[0].velocity, ship[-1].velocity)          # ship[i] -> State
```

```
    ship.reset()                                                    # back to the initial
    ↪ state
```

OUTPUT

```
61 states | v(0) = 8.0 kt | v(60 s) = 10.3326133909287 kt
```

Simulation step `step_interval` sets the granularity of the simulation. Position, range to the sensor and velocity are sampled once per step, and the channel response, Doppler factor and cavitation spectrum are updated from them. Steps of 1 s are suitable for ships. Fast maneuvers or short ranges call for smaller steps.

5.3 Controlling the dynamics over time

The scenario advances with `scenario.simulate(n_steps)`, which moves *all* elements. Between two calls, the latest state of any element can be changed, and the following steps start from it. An arbitrary maneuver therefore becomes a control loop. Three helper functions keep the code readable:

examples_en/ex05_dynamics.py

```
def set_velocity(element: lps_dynamic.Element, velocity: lps_dynamic.Velocity) -> None:
    """ Changes the element velocity from the current instant on. """
    element.state_list[-1].velocity = velocity


def set_acceleration(element: lps_dynamic.Element,
                     acceleration: lps_dynamic.Acceleration) -> None:
    """ Changes the element acceleration from the current instant on. """
    element.state_list[-1].acceleration = acceleration


def course_and_speed(course_deg: float, speed: lps_qty.Speed) -> lps_dynamic.Velocity:
    """ Velocity from a nautical course (0° = north, clockwise) and magnitude. """
    return lps_dynamic.Velocity.polar2d(speed, lps_qty.Bearing.ncw_deg(course_deg))
```

A four-phase maneuver (constant course, acceleration, turn and slowdown):

examples_en/ex05_dynamics.py

```
    sonar = lps_sonar.Sonar.hydrophone(sensitivity=lps_qty.Sensitivity.db_v_p_upa(-170))

    ship = lps_noise.Ship(
        ship_id="Target",
        propulsion=lps_noise.CavitationNoise(lps_noise.ShipType.FISHING, seed=2),
        draft=lps_qty.Distance.m(5),
        initial_state=lps_dynamic.State(
            position=lps_dynamic.Displacement(lps_qty.Distance.m(-250),
    ↪ lps_qty.Distance.m(100)),
            velocity=course_and_speed(90, lps_qty.Speed.kt(6))))      # course east at 6 kt

    scenario = lps_scenario.Scenario(step_interval=lps_qty.Time.s(1), channel=None)
    scenario.add_sonar("main", sonar)
```

```
    scenario.add_noise_container(ship)

    # Phase 1: 30 s on a constant course
    scenario.simulate(30)

    # Phase 2: accelerate from 6 to 10 kt (a = 0.1 m/s² along the motion)
    heading = ship[-1].velocity.get_azimuth()
    set_acceleration(ship, lps_dynamic.Acceleration.polar2d(
        lps_qty.Acceleration.m_s2(0.1), heading))
    scenario.simulate(20)

    # Phase 3: 90° turn to starboard (to the south) at 3°/s, keeping the speed
    set_acceleration(ship, lps_dynamic.Acceleration(lps_qty.Acceleration.m_s2(0),
                                                    lps_qty.Acceleration.m_s2(0)))
    course = 90
    for _ in range(30):
        course += 3
        set_velocity(ship, course_and_speed(course, ship[-1].velocity.get_magnitude()))
        scenario.simulate(1)

    # Phase 4: drop to 3 kt instantly and keep going for 40 s
    set_velocity(ship, course_and_speed(course, lps_qty.Speed.kt(3)))
    scenario.simulate(40)
```

OUTPUT

```
simulated steps: 120
t=  0 s  pos=( -250.0,   100.0) m  |v|= 6.00 kt  course=  90.0°
t= 30 s  pos=( -157.4,   100.0) m  |v|= 6.00 kt  course=  90.0°
t= 50 s  pos=(  -75.7,   100.0) m  |v|= 9.89 kt  course=  93.0°
t= 80 s  pos=(   18.9,     0.3) m  |v|= 3.00 kt  course= 180.0°
t=120 s  pos=(   18.9,   -61.4) m  |v|= 3.00 kt  course= 180.0°
```

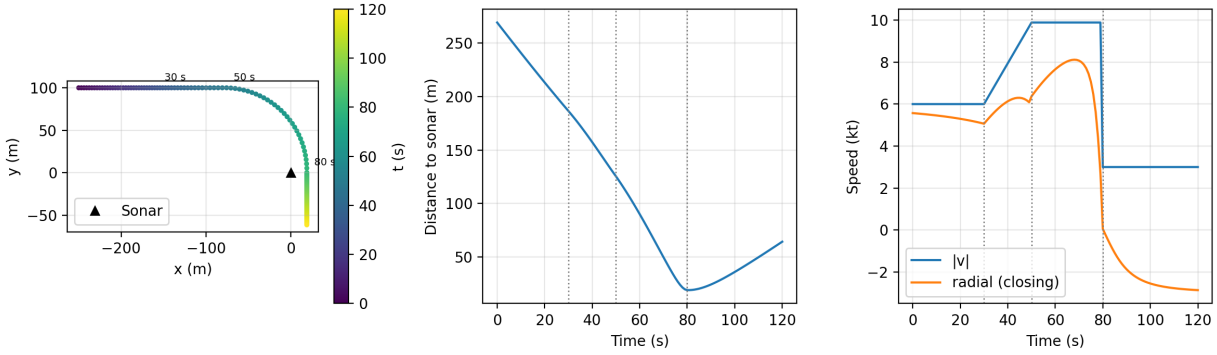


Figure 5.1: The resulting maneuver: trajectory colored by time, range to the sonar and speed (magnitude and radial component). Dotted lines mark the start of each phase.

Warning Changing `state_list[-1]` modifies the state at the current instant, which is already part of the trajectory. The change takes effect from that sample, not the next one. Also remember that the speed is capped by `max_speed`, which for a `Ship` comes from its propulsion.

5.4 Ready-made scenario plots

examples_en/ex05_dynamics.py

```
scenario.geographic_plot(fig_path("ex05_geographic.png"))        # top view
scenario.relative_distance_plot(fig_path("ex05_distance.png"))   # distance vs time
scenario.velocity_plot(fig_path("ex05_velocity.png"))            # |v| vs time
scenario.relative_velocity_plot(fig_path("ex05_rel_velocity.png"))  # radial speed
scenario.bearing_plot(fig_path("ex05_bearing.png"))              # true bearing
scenario.relative_bearing_plot(fig_path("ex05_rel_bearing.png"))  # relative bearing
```

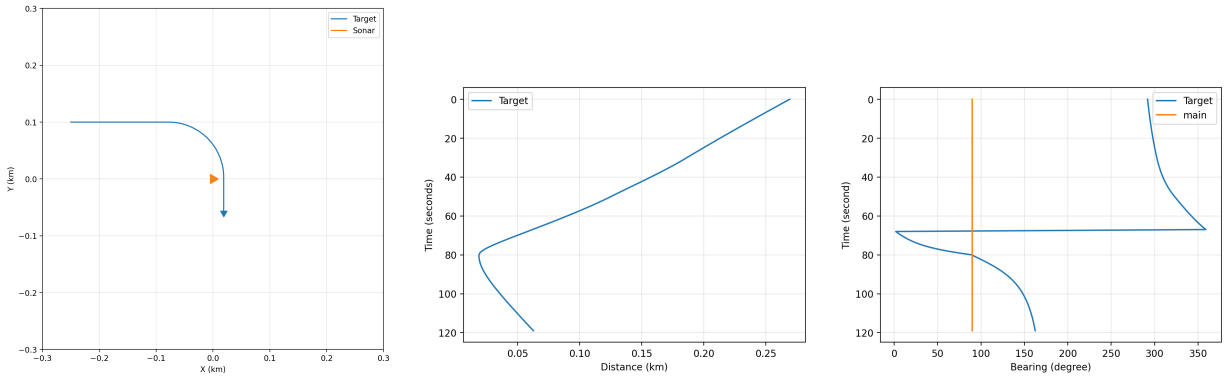


Figure 5.2: `geographic_plot`, `relative_distance_plot` and `bearing_plot` for the maneuver above. In the last two, time grows downwards, as in a sonar waterfall.

5.5 Standardized dynamics

To generate many samples with controlled geometries, the `lps_synthesis.database.dynamic` module defines three dynamic types based on the desired closest range. The class computes the initial states of the ship and the sonar:

examples_en/ex05_dynamics.py

```
import lps_synthesis.database.dynamic as syndb_dynamic

step, n_steps = lps_qty.Time.s(1), 60
dynamic = syndb_dynamic.SimulationDynamic(
    dynamic_type=syndb_dynamic.DynamicType.CPA_IN,   # FIXED_DISTANCE, CPA_IN, CPA_OUT
    shortest=lps_qty.Distance.m(80),                 # closest distance to the sonar
    approaching=True)

ship_state = dynamic.get_ship_initial_state(speed=lps_qty.Speed.kt(8),
                                            step_interval=step,
                                            simulation_steps=n_steps)
```

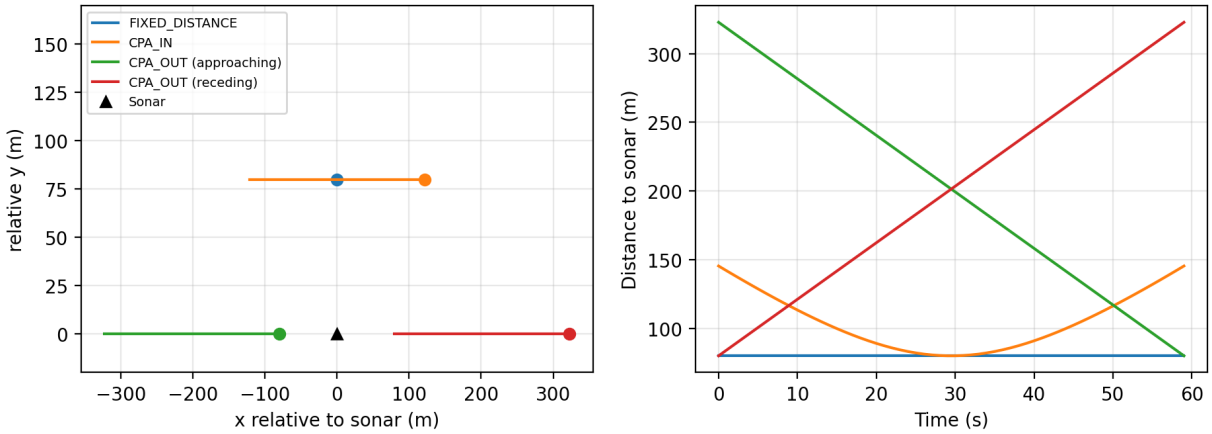


Figure 5.3: `SimulationDynamic` types for $d_{\text{min}} = 80\,\text{m}$ and 8 kt over 60 s. `FIXED_DISTANCE`: the sonar follows the ship. `CPA_IN`: the closest point of approach is in the middle of the window. `CPA_OUT`: the CPA is at the end (approaching) or at the start (receding).

CHAPTER 6

Complete scenario and `.wav` export

This chapter puts all the pieces together: the environment and channel from Chapter 2, two ships (one with tones at different hull positions), a buoy transmitting LFM pulses, a hydrophone and a four-sensor array.

6.1 Setup

examples_en/ex06_scenario.py

```
fs = lps_qty.Frequency.khz(16)
step = lps_qty.Time.s(1)
n_steps = 45

environment = lps_env.Environment(rain_value=lps_env.Rain.NONE,
                                  sea_value=lps_env.Sea.STATE_2,
                                  shipping_value=lps_env.Shipping.LEVEL_2,
                                  seed=10)
channel = build_channel()                       # Ch. 2 (read from cache)

scenario = lps_scenario.Scenario(step_interval=step,
                                 channel=channel,
                                 environment=environment)
```

Warning If `environment` is omitted, `Scenario` draws an `Environment.random()`. It does *not* turn background noise off. To simulate without ambient noise, call `sonar.get_data(..., environment=None)` directly (Section 6.4).

examples_en/ex06_scenario.py

```
scenario.add_sonar("hydrophone", lps_sonar.Sonar.hydrophone(
    sensitivity=lps_qty.Sensitivity.db_v_p_upa(-180),
    signal_conditioner=lps_sonar.IdealAmplifier(gain_db=50)))

scenario.add_sonar("array", lps_sonar.Sonar.planar(
    n_staves=4,
    spacing=lps_qty.Distance.m(0.5),
    sensitivity=lps_qty.Sensitivity.db_v_p_upa(-180),
    signal_conditioner=lps_sonar.IdealAmplifier(gain_db=50),
    initial_state=lps_dynamic.State(
        position=lps_dynamic.Displacement(lps_qty.Distance.m(0),
↪ lps_qty.Distance.m(20)))))
```

examples_en/ex06_scenario.py

```
# Target 1: bulker passing 60 m from the sonar (CPA mid-recording)
bulker = lps_noise.Ship(
    ship_id="Bulker",
    propulsion=lps_noise.CavitationNoise(
        ship_type=lps_noise.ShipType.BULKER, n_blades=4, n_shafts=1,
        length=lps_qty.Distance.m(180), cruise_speed=lps_qty.Speed.kt(12),
        cruise_rotacional_frequency=lps_qty.Frequency.rpm(90),
        max_speed=lps_qty.Speed.kt(16), seed=1),
    draft=lps_qty.Distance.m(10),
    initial_state=lps_dynamic.State(
        position=lps_dynamic.Displacement(lps_qty.Distance.m(-115),
↪ lps_qty.Distance.m(60)),
        velocity=lps_dynamic.Velocity(lps_qty.Speed.kt(10), lps_qty.Speed.kt(0))),
    seed=1)
bulker.add_source(lps_noise.NarrowBandNoise(lps_qty.Frequency.hz(60),
↪ amp_db_p_upa=178))
bulker.add_source(lps_noise.NarrowBandNoise(
    lps_qty.Frequency.hz(420), amp_db_p_upa=168,
    rel_position=lps_dynamic.Displacement(lps_qty.Distance.m(40),
↪ lps_qty.Distance.m(0))))

# Target 2: fishing boat slowly moving away, with an oscillating generator tone
fishing = lps_noise.Ship(
    ship_id="Fishing",
    propulsion=lps_noise.CavitationNoise(lps_noise.ShipType.FISHING, seed=4),
    draft=lps_qty.Distance.m(5),
    initial_state=lps_dynamic.State(
        position=lps_dynamic.Displacement(lps_qty.Distance.m(30),
↪ lps_qty.Distance.m(-80)),
        velocity=lps_dynamic.Velocity.polar2d(lps_qty.Speed.kt(4),
                                              lps_qty.Bearing.ncw_deg(160))),
    seed=4)
fishing.add_source(lps_noise.NarrowBandNoise.with_sine_fm_modulation(
    frequency=lps_qty.Frequency.hz(900), amp_db_p_upa=150,
    oscilation_freq=lps_qty.Frequency.hz(0.1),
    deviation_freq=lps_qty.Frequency.hz(15)))

scenario.add_noise_container(bulker)
scenario.add_noise_container(fishing)

# Standalone source (no ship): buoy transmitting a chirp, stationary at 150 m
scenario.add_noise_source(
    lps_noise.NarrowBandNoise.with_fm_chirp(
        amp_db_p_upa=160,
        start_frequency=lps_qty.Frequency.khz(2.5),
        end_frequency=lps_qty.Frequency.khz(3.5),
        tx_interval=lps_qty.Time.s(10),
        tx_duration=lps_qty.Time.s(1)),
    initial_state=lps_dynamic.State(
        position=lps_dynamic.Displacement(lps_qty.Distance.m(-120),
↪ lps_qty.Distance.m(-90),
                                          lps_qty.Distance.m(5))))
```

`add_noise_source` is a shortcut that creates a `NoiseContainer` with a single source, useful for buoys, *pingers* and biological sources. The depth of that source is the z coordinate of its initial

position.

6.2 Simulation

examples_en/ex06_scenario.py

```
scenario.simulate(n_steps)
# With more than one sonar, the id is appended to the name: ex06_geographic_<id>.png
scenario.geographic_plot(fig_path("ex06_geographic_"))
scenario.relative_distance_plot(fig_path("ex06_distance_"))
```

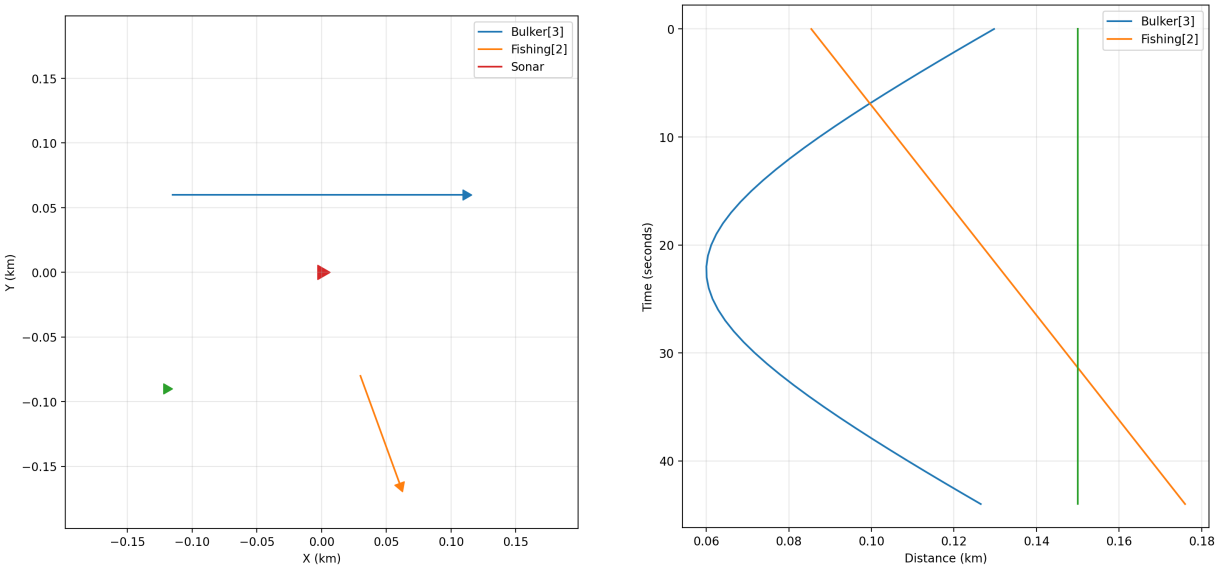


Figure 6.1: Scenario geometry and range from each source to the hydrophone. With more than one sonar, each plot is generated per sonar and the file name is `<filename><sonar_id>.png`, which is why the example passes a prefix ending in _.

6.3 Generating audio and exporting .wav

examples_en/ex06_scenario.py

```
audio = scenario.get_sonar_audio("hydrophone", fs=fs)       # int16, (n_samples,
↪ n_channels)
lps_signal.save_wav(audio, fs, wav_path("scenario_hydrophone.wav"))

audio_array = scenario.get_sonar_audio("array", fs=fs)  # 4 channels -> multichannel
↪ .wav
wavfile.write(wav_path("scenario_array.wav"), int(fs.get_hz()), audio_array)
```

OUTPUT

```
hydrophone: (719946, 1) int16 | duration = 45.0 s
array     : (719946, 4) int16
volts     : (719946, 1) float64 | rms = 387.7 mV
```

```
full scale used: 50.6 %
total example time: 382 s
```

`get_sonar_audio` returns an `int16` array of shape `(n_samples, n_sensors)`, which is already a valid multichannel `.wav`. Both `scipy.io.wavfile.write` and `lps_sp.signal.save_wav` write it unchanged. `save_wav` normalizes automatically when it receives a floating-point array.

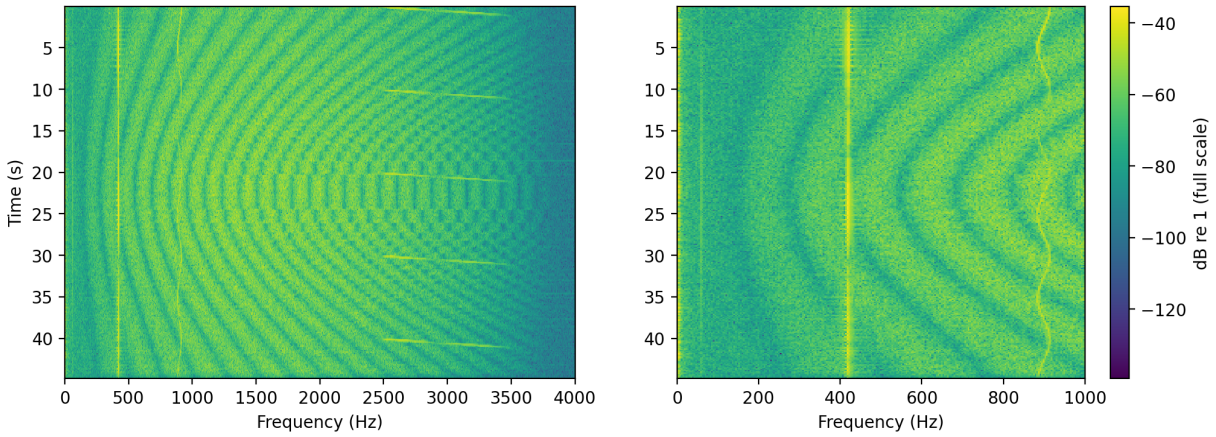


Figure 6.2: Hydrophone spectrogram (LOFAR): broadband noise of both ships with channel interference fringes, the bulker's tones (60 Hz, 420 Hz), the fishing boat's slow-FM tone (900 Hz) and the buoy's LFM pulses (2.5–3.5 kHz, every 10 s). Right: detail up to 1 kHz.

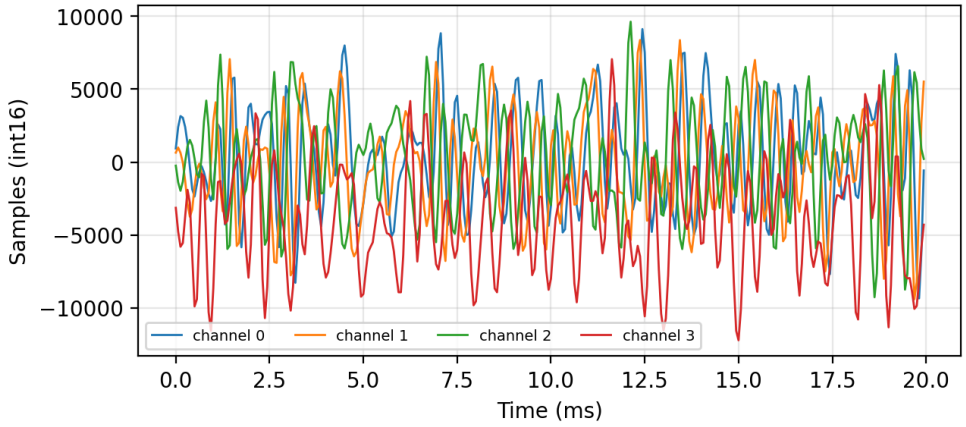


Figure 6.3: 20 ms of the four array channels: the small phase differences between channels carry the direction-of-arrival information.

6.4 Lower-level access

`get_sonar_audio` is a shortcut for `NoiseCompiler` + `Sonar.get_data`. Calling them directly lets you get the signal in volts (before the ADC), turn the background noise off (`environment=None`) or parallelize over sources and sensors:

examples_en/ex06_scenario.py

```
# Lower-level access: signal in volts (before the ADC) and parallel processing
compiler = lps_noise.NoiseCompiler(scenario.noise_containers, fs=fs, parallel=True)
```

```
volts = scenario.sonars["hydrophone"].get_data(noise_compiler=compiler,
                                               channel=scenario.channel,
                                               environment=scenario.environment,
                                               without_digitalization=True,
                                               parallel=True)
lps_signal.save_normalized_wav(volts[:, 0], fs,
↪ wav_path("scenario_hydrophone_norm.wav"))
```

6.5 Reproducibility and performance

- **Seeds.** `Environment`, `CavitationNoise`, `Ship` and the Brownian variants accept a `seed`. Without one, the seed is `id(object)` and every run produces different signals. For reproducible datasets, fix every seed.
- **Reusing a scenario.** `scenario.reset()` brings every element back to its initial state. You can, for example, swap the sonar and simulate again with the same geometry.
- **Cost.** Propagation is a time-varying convolution computed sample by sample. The cost is proportional to (samples) $\times$ (impulse response length) $\times$ (source groups) $\times$ (sensors). Use `parallel=True`, reduce `max_distance` (shorter response) and group sources at the same hull position.
- **Channel limits.** Keep every source within the channel's `max_distance`. Beyond it, the response of the last point is repeated and spreading loss stops increasing.

CHAPTER 7

Catalogs and datasets

The `lps_synthesis.database` package automates large-scale dataset generation. It combines three catalogs:

- `ShipCatalog`: ships drawn from `database/data/ship_info.csv`, with propulsion, draft, speed and narrowband components consistent with their class;
- `SimulationDynamic`: the dynamics from Chapter 5;
- `AcousticScenario`: (`Location`, `Season`) pairs with channel and background noise derived from real oceanographic data (20 predefined locations).

7.1 Ships from the catalog

examples_en/ex07_database.py

```
catalog = syndb_ship.ShipCatalog(n_samples=4, seed=42)    # draws from ship_info.csv
dynamics = syndb_dynamic.SimulationDynamic.rand_catalog(n_samples=4, seed=42)

step, n_steps = lps_qty.Time.s(1), 60
ships = [info.make_ship(dynamic=dyn, step_interval=step, simulation_steps=n_steps)
         for info, dyn in zip(catalog, dynamics)]
```

examples_en/ex07_database.py

```
    for info, dyn, ship in zip(catalog, dynamics, ships):
        print(f"{info.ship_type!s:<14} L={info.length.get_m():5.1f} m "
              f"v={info.current_speed.get_kt():4.1f} kt | {dyn} | "
              f"{len(ship.noise_sources)} sources")

    df = catalog.to_df()            # DataFrame with all drawn parameters
```

OUTPUT

```
tug            L= 35.0 m v= 7.7 kt | FIXED_DISTANCE (d_min=131 m) | 7 sources
tanker         L=122.0 m v= 7.7 kt | FIXED_DISTANCE (d_min=85 m) | 4 sources
fishing        L= 26.0 m v= 9.2 kt | FIXED_DISTANCE (d_min=144 m) | 8 sources
tug            L= 32.0 m v= 8.8 kt | FIXED_DISTANCE (d_min=119 m) | 4 sources
columns: CATALOG_ID, SEED, CLASS, SHIP_TYPE, MCR_PERCENT, CRUISING_SPEED_KT, ROTACIONAL_FREQUENCY,
    ↪ LENGTH_M ...
```

7.2 Real locations and dataset generation

The snippet below depends on the environmental data (`/data/ambiental`) and on precomputed channels (`/data/channel`), and was *not* run when generating this manual:

not executed

```
import lps_synthesis.database.scenario as syndb_scenario
import lps_synthesis.environment.acoustic_site as lps_site
import lps_synthesis.database.database as syndb

site = syndb_scenario.AcousticScenario(syndb_scenario.Location.GUANABARA_BAY,
                                       lps_site.Season.SUMMER)
channel = site.get_channel()          # local SSP (WOA18) + bathymetry + sediment
environment = site.get_env()          # typical rain and sea state for the season

# Full dataset: ship catalog x locations x dynamics
db = syndb.IEMANJA(n_ships_conditions=50, n_fixed_scenarios=2, n_random_scenarios=2)
db.export("output/")                  # catalog CSVs
db.synthesize(output_dir="output/", sonar=lps_sonar.Sonar.hydrophone(...),
              sample_frequency=lps_qty.Frequency.khz(16), step_interval=lps_qty.Time.s(1),
              simulation_steps=60, global_attenuation_db=0)
```

The script `lps_libraries/acoustic_synthesis/apps/make_database.py` is the command-line interface for this workflow.

APPENDIX A

Quick reference

Task	Code
Manual SSP	`d = Description(); d.add(Distance.m(z), Speed.m_s(c)); d.add(Distance.m(H), BottomType.SAND)`
Random SSP	`Description.get_random(seed=0)`
Channel	`Channel(QueryConfig(description=d, sensor_depth=..., source_depths=[...], max_distance=..., frequency_range=(...)), model=Oases(), channel_dir=..., hash_id="...")`
Background noise	`Environment(Rain.LIGHT, Sea.STATE_3, Shipping.LEVEL_3, seed=0)`
Ship	`Ship("id", propulsion=CavitationNoise(ShipType.BULKER, seed=0), draft=Distance.m(10), initial_state=State(...))`
Quick ship	`Ship.by_type(ShipType.FISHING, seed=0)`
Tone	`ship.add_source(NarrowBandNoise(Frequency.hz(60), amp_db_p_upa=170))`
Standalone source	`scenario.add_noise_source(source, initial_state=State(...))`
Hydrophone	`Sonar.hydrophone(sensitivity=Sensitivity.db_v_p_upa(-180))`
Line array	`Sonar.planar(n_staves=8, spacing=Distance.m(0.5), sensitivity=...)`
Cylindrical array	`Sonar.cylindrical(n_staves=16, radius=Distance.m(1), sensitivity=...)`
Scenario	`s = Scenario(step_interval=Time.s(1), channel=ch, environment=env)`
Simulate	`s.add_sonar("main", sonar); s.add_noise_container(ship); s.simulate(60)`
Change dynamics	`ship.state_list[-1].velocity = Velocity(...); s.simulate(n)`
Audio	`audio = s.get_sonar_audio("main", fs=Frequency.khz(16))`
Export	`lps_sp.signal.save_wav(audio, fs, "output.wav")`
Plots	`s.geographic_plot(f); s.relative_distance_plot(f); s.bearing_plot(f)`

APPENDIX B

Known pitfalls

Issues observed in the code while writing this manual. Some are design limitations; others are probably bugs to be fixed.

1. **Channel cache collision.** `QueryConfig` is declared as `@dataclass(slots=True)`, but `Hashable` serializes `self.__dict__`, which does not exist on classes with `__slots__`. As a result, `hash(query)` is constant (`822076307648613867`) for any SSP, geometry or band. Since the cache file name is `{hash_id}_{hash(query)}_{model}.pkl`, only `hash_id` tells channels apart. Workaround: a unique `hash_id` per channel. Suggested fix: drop `slots=True` or implement `_get_params` in `QueryConfig`.
2. **Outdated examples.** Some scripts in `lps_libraries/acoustic_synthesis/example/` use APIs that no longer exist (e.g. `PredefinedChannel.CYLINDRICAL`; only `SPHERICAL` exists today). Prefer the examples in `docs/manual/examples_en/`.
3. **`Ship` without `propulsion`** fails (Chapter 3).
4. **Brownian variants** accumulate per-sample deviations (Chapter 3).
5. **Sonar depth** comes from the channel, not from the position (Chapter 4).
6. **Source depth** is rounded to the closest computed depth in `source_depths`. The grid is regular, with a step equal to the GCD of the differences, so irregular lists can produce many more depths than expected.
7. **Beyond the channel range**, the response of the last point is repeated.
8. **`ShipType.RESEARCH`** has the same value as `GOVERNMENT` and is therefore an alias for it in Python.
9. **`Description` JSON** loses custom layers (Chapter 1).
10. **Plots with several sonars.** The `Scenario` `*_plot` methods write to `<filename><sonar_id>.png` when there is more than one sonar (e.g. `geo.pngmain.png`). Pass a prefix, not a file name with an extension.
11. **`Scenario` without `environment`** uses a random environment, not silence.